Testbericht 17 – Schablonenschrift „cad stencil mono“

Plan 17.7, Schritt 3b: eigene monospaced Schablonenschrift als Standardschrift von `text()` · cad-font (`stencil.rs`), cad-eval (`text()`-Parameter), Tests in cad-font, cad-eval, cad-engine, compat-tests, Playwright; Korrektur Playwright C16/C20 · 24.10.2026 · Branch worktree-agent-a84f54f259309a87b · Commit 436b868 · rustc 1.97.0 (2d8144b78 2026-07-07) · 4 Kerne

Glyphen ohne Loch	cargo test --workspace	clippy · fmt	Vitest · Playwright	Schild, Kernel nativ
125 / 125	**529 / 529**	**sauber · sauber**	**221 · 106 / 106**	**1.791 ms**
Regular und Bold; 8 Parameterkombinationen im Test	davon 13 neu/geändert	-D warnings	neu: K01–K03; C16 behoben	Allerta Stencil 3.419 ms

Wasm cad_wasm
+41,5 KiB
gzip +15,0 KiB

1 Auftrag und Ergebnis

Plan 17.4 sieht als Standardschrift eine eigene, sehr einfache monospaced Schablonenschrift aus Geraden und Bögen vor, mit Stegen, sodass keine Inseln herausfallen. Umgesetzt:

- **„cad stencil mono“** (Regular, Bold) in `crates/cad-font/src/stencil.rs`: 125 Glyphen – der ganze Pflicht-Zeichensatz `cad_font::REQUIRED_CHARS` ohne Rückfall, dazu `’ ” $`. Jede Glyphe ist eine Folge von Mittellinien (Geraden, tangentiale Kreisbögen), gezeichnet mit einem Stift der Strichstärke (flache Enden, Gehrung), plus Punkte, minus Stege, an Grund- bzw. Versallinie beschnitten. Die Kontur rechnet `cad_kernel::region2` exakt aus (Stücke der Geraden und Bögen); STEP und Kernel sehen Geraden und Kreise.
- **Keine Löcher:** jede Glyphe, zum Profil gerendert, hat 0 Löcher – Innenräume von O, A, B, D, P, R, 0, 4, 6, 8, 9, Ä, ö, @, %, &, §, °, Ø … sind über Stege mit dem Außenbereich verbunden. Geprüft in cad-font für 8 Kombinationen von Strichstärke und Stegbreite (Regionen ohne Uhrzeigersinn-Schleifen, geschlossene Schleifen, keine Splitter, keine Überlappungen) und in der Engine für jede Glyphe über `cad_engine::region_2d`.
- **Parameter** `text(…, stroke_width = …, bridge_width = …)` in mm: Strichstärke Standard 0,14·`size` (Bold 0,2·`size`), höchstens 0,2·`size`; Stegbreite Standard 0,1·`size`, 0 (keine Stege – geschlossene Innenräume wie eine normale Schrift) oder 0,03…0,2·`size`. Werte außerhalb: Warnung am Argument, begrenzt. Bei anderen Schriften: Warnung „wirkt nur mit …“.
- **Metrik** wie die übrigen Schriften: em = `size`/0,72, Versalhöhe = `size` (720/1000 em), Vorschub 600/1000 em × 1000/1024 – genau wie JetBrains Mono, sodass Texte gleich breit bleiben wie mit der bisherigen Standardschrift.
- **Standardschrift:** `text()` ohne `font` und mit unbekannter Familie nutzt die Schablonenschrift (vorher JetBrains Mono); fehlende Zeichen (z. B. Kyrillisch) kommen weiter aus JetBrains Mono gleichen Gewichts.
- Nebenbei: Playwright **C16** (achsgebundenes Ziehen, Phase B), das zuletzt auch einzeln scheiterte, ist behoben – die Ursache lag im Test (Griffe der vorigen Cursorzeile, Abschnitt 10).

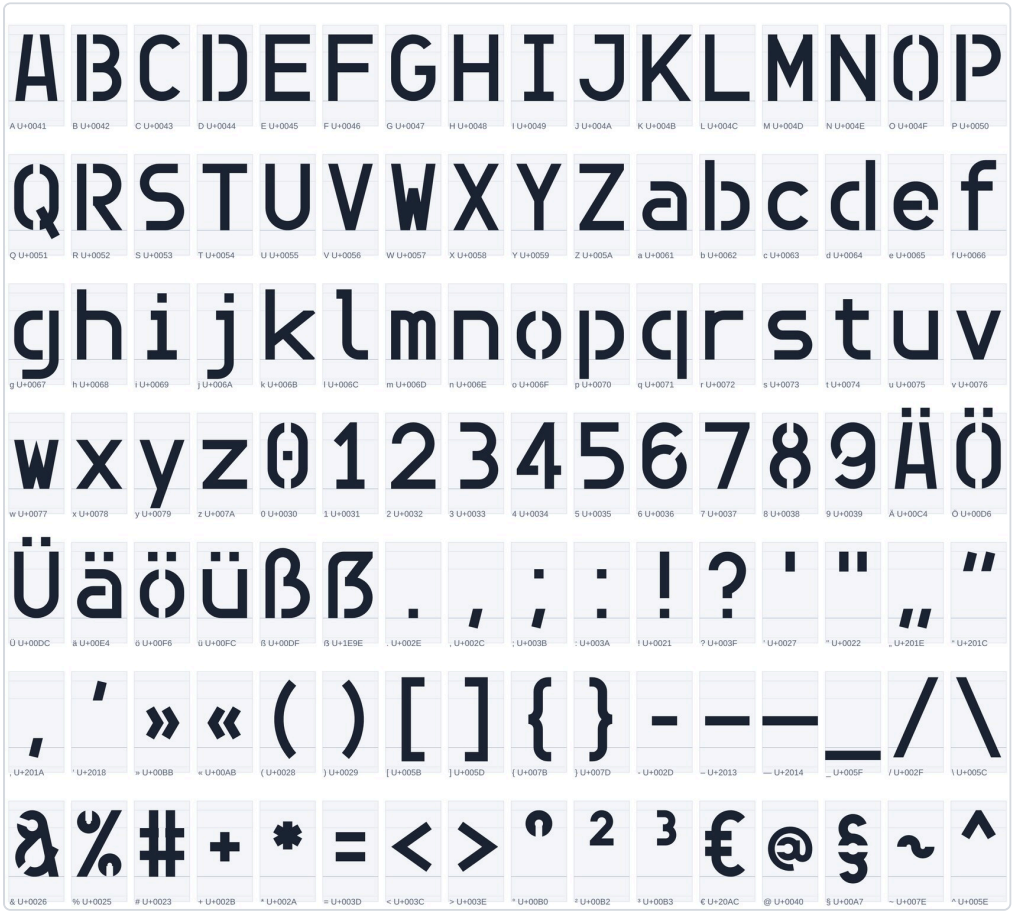
2 Entscheidungen

Frage	Entscheidung
Form der Schrift	Rust-Daten statt Schriftdatei: je Zeichen eine kleine Zeichenfunktion über benannten Lagen (Mittellinien links/rechts/Mitte, Grund-, x-, Versal-, Ober-, Unterlänge, abhängig von der Strichstärke), Bausteine Stadion, Bauch, C-Bogen, Punkte, Kommas. Kein TTF-Build, keine Lizenzfrage; Wasm-Zuwachs nur Code (Abschnitt 8).

Strich	`region2::stroke` (aus Schritt 2) mit flachen Enden und Gehrung: rechte Winkel bleiben eckig, Enden sind gerade Schnitte (Schablonen-Anmutung). Freie Enden laufen bis zur äußeren Linie (Grundlinie, Versalhöhe), Stöße enden auf der Mittellinie des anderen Strichs – so entstehen keine Kerben. Diagonalen (A, K, M, N, V, W, X, Y, Z, 1, 2, 4, 7, `/`) werden über die Linie verlängert und an Grund-/Versallinie beschnitten (gerade Füße, flache Spitzen).
Stege	Ein Steg ist ein Band der Stegbreite quer über einen Strich (Länge 1,3 halbe Striche zu jeder Seite), von der Glyphe abgezogen. Regeln: Stamm getrennt vom Bauch bei B, D, P, R, b, d, g, p, q (Stege am Stamm), runde Formen oben und unten (O, Q, 0, 8, o), offenes A (Beine an der Versallinie um die Stegbreite auseinander), e und a am Querstrich bzw. am Bauch neben dem Stamm, 4 am Querstrich vor dem Stamm, 6/9 schräg am Bauch, Ringe (°, %, Ø, @, &, §) an einer Seite. ß/ß sind offen (Lücke ≥ Stegbreite zum Stamm).
Parameter	`stroke_width` und `bridge_width` in mm (für Druck und Fräsen sind absolute Maße gefragt: „Steg mindestens zwei Düsenbreiten“); ohne Angabe proportional zu `size`. Stil Bold = Standard-Strichstärke 0,2·`size`. Die Grenzen kommen aus dem Test der Parameterfläche: oberhalb 0,2·`size` wachsen Striche von a, e, #, @ zusammen und schließen Innenräume; Stege unter 0,03·`size` schließen schwere Striche wieder.
Metrik	1000 Einheiten je em, Versalhöhe 720, x-Höhe 530, Oberlänge 760, Unterlänge −210, Vorschub 600, Seitenrand 60 (Glyphenbox 480 breit). Senkrechter Text: Vorschub 1000, Glyphe mittig auf dem Stift (wie der Shaper bei TrueType).
Standardschrift und OpenSCAD	OpenSCAD kennt die Schrift nicht (es nimmt ohne `font` die fontconfig-Standardschrift, meist Liberation Sans): ein Modell ohne `font` sieht in OpenSCAD anders aus. Die Kompatibilitätstests hängen davon nicht ab (Abschnitt 9); wer gleiche Maße wie OpenSCAD braucht, gibt eine mitgelieferte TrueType-Schrift an. Unbekannte Familien → Standardschrift mit Warnung (vorher JetBrains Mono).

3 Zeichentafel

Alle 125 Glyphen direkt aus der Schrift (Beispielprogramm `cargo run -p cad-font --example stencil_chart`): Zelle = Vorschub, Linien Grundlinie, x-Höhe, Versalhöhe, Unterlänge. Eine Glyphe mit Loch wäre rot.



cad stencil mono Regular (Strich 0,14·size, Steg 0,1·size)



cad stencil mono Bold (Strich 0,2·size, Steg 0,1·size)

Teile je Glyphe (Regular): 88× 1 Teil, 27× 2 Teile, 8× 3 Teile, 2× 4 Teile; Löcher: Regular 0, Bold 0.

4 Parameter

Dieselben acht Zeichen mit verschiedenen Strich- und Stegbreiten (Anteile von `size`). `bridge_width = 0` schaltet die Stege ab: dann haben O, B, 0, 8 … wieder Innenräume (rot = Glyphe mit Loch, so gewollt).



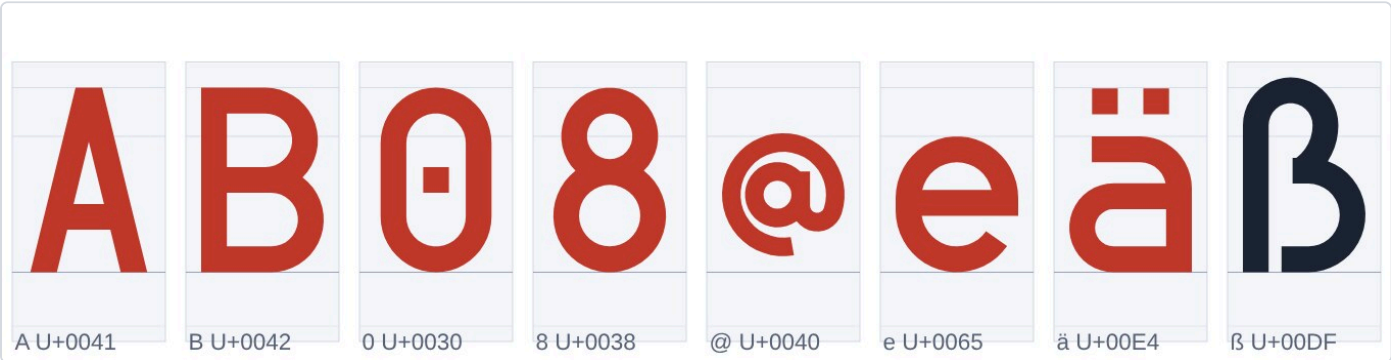
Strich 0,06 · Steg 0,04



Standard: Strich 0,14 · Steg 0,1



Strich 0,2 · Steg 0,2 (Grenzen)



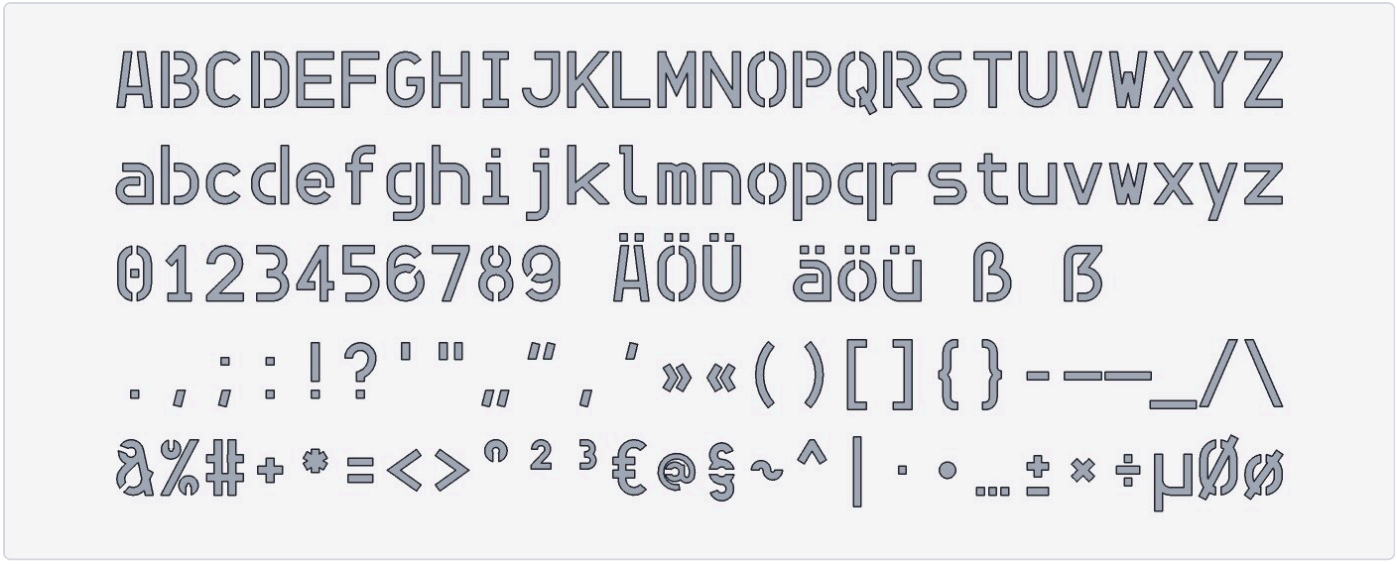
Strich 0,14 · ohne Stege (bridge_width = 0)

Warnungen bei Werten außerhalb der Grenzen und bei anderen Schriften:

```
cases/warn.scad:1:37: Warnung (Auswertung): text: `stroke_width` = 3 mm bei size = 10: höchstens 2 mm
cases/warn.scad:1:55: Warnung (Auswertung): text: `bridge_width` = 0.1 mm bei size = 10: mindestens 0.3 mm
(oder 0)
cases/warn.scad:2:42: Warnung (Auswertung): text: `bridge_width` wirkt nur mit der Schrift „cad stencil mono“
cases/warn.scad:3:18: Warnung (Auswertung): Schrift „Comic Sans“ nicht vorhanden, verwendet wird die
Standardschrift cad stencil mono (mitgeliefert: cad stencil mono, JetBrains Mono, Inter, Allerta Stencil;
eigene Schrift: font = "datei.ttf")
cases/warn.scad: 0 Fehler, 4 Warnungen
```

5 Vergleich mit Allerta Stencil

Alphabet-Tafeln über die CLI (`text(…, size = 8)`, 0,6 mm extrudiert, von oben): die Schablonenschrift hat alle Zeichen selbst, Allerta Stencil (proportional) 20 davon nicht (aus JetBrains Mono, nicht schabloniert, Bericht 16).



cad stencil mono Regular



cad stencil mono Bold



Allerta Stencil – ß „ , { } _ # ° ² ³ § ~ ^ · • … ± µ Ø ø aus JetBrains Mono

Gleiches Schild in beiden Schriften



cad stencil mono (Standard) – Volumen 6281.4993 mm³



Allerta Stencil – Volumen 6294.8797 mm³

```
// Schild mit erhabenem Text (Bericht 17): stencil
color("white") cube([100, 30, 2]);
color("black") translate([50, 17, 2]) linear_extrude(1) text("WERKSTATT 3", size = 8, halign = "center");
color("black") translate([50, 5, 2]) linear_extrude(1) text("ZUTRITT NUR MIT BRILLE", size = 4.2, halign =
"center");
```

Merkmal	cad stencil mono	Allerta Stencil
Breite	monospaced, 0,814·size je Zeichen (wie JetBrains Mono)	proportional, mit Kerning
Kurven	Geraden und Kreisbögen (Bögen > 90° im Kernel geteilt)	quadratische Bézierkurven → je Flanke eine B-Spline
Pflicht-Zeichensatz	vollständig, alle schabloniert	20 Zeichen fehlen (Rückfall, ohne Stege)
Strich, Steg	einstellbar (`stroke_width`, `bridge_width`), Bold	fest
Lizenz	Quelltext des Projekts	SIL OFL 1.1, Reserved Font Name (nur unverändert)

6 Beispiel: Schablone

`examples/spec/schablone.scad`: Text aus einer 1 mm dünnen Platte geschnitten. Ohne Stege fielen die Innenteile von A, 0, 8, B, 9 heraus; mit der Schablonenschrift hängen sie an der Platte.

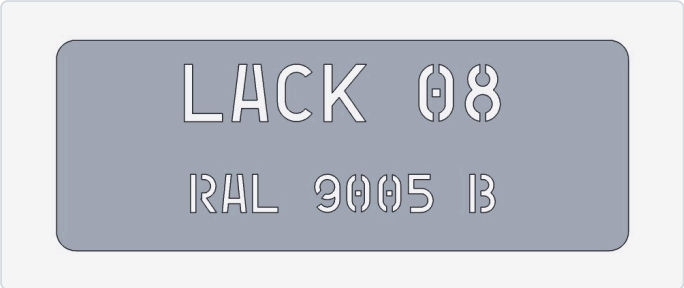
```
// Zweck: Sprühschablone mit der Standardschrift „cad
stencil mono“ (Plan 17.4, Bericht 17) – der
// Text ist aus einer dünnen Platte ausgeschnitten;
die Stege der Schrift halten die Innenteile
// von O, A, B, 0, 8 … an der Platte, nichts fällt
heraus. Ohne `font` gilt die Schablonenschrift;
// `stroke_width` (Strichstärke) und `bridge_width`
(Stegbreite) in mm, `bridge_width = 0` ohne Stege.

breite = 120;
hoehe = 44;
dicke = 1;
r = 4;              // Eckenradius
groesse = 12;       // Versalhöhe
strich = 1.7;       // Strichstärke (Standard 0,14 ·
size)
steg = 1.2;         // Stegbreite (Druck: mindestens
zwei Düsenbreiten)

difference() {
    linear_extrude(dicke) offset(r = r) offset(delta
= -r) square([breite, hoehe]);
    translate([breite / 2, 27, -1])
linear_extrude(dicke + 2)
        text("LACK 08", size = groesse, halign =
"center", stroke_width = strich, bridge_width =
steg);
    translate([breite / 2, 8, -1])
linear_extrude(dicke + 2)
        text("RAL 9005 B", size = 8, halign =
"center", bridge_width = steg);
}
```



gerendert (iso)



von oben

Gedruckt: G-Code aus PrusaSlicer

Ein echter Druck ist in dieser Umgebung nicht möglich; stattdessen: STL exportiert (`cad export`), mit PrusaSlicer 2.7 (Standardprofil, Schicht 0,2 mm, Düse 0,4 mm) zu G-Code geschnitten und die oberste Schicht aus dem G-Code gezeichnet (`tools/report/report17-gcode.py`): jede Insel der Platte ist mit dem Rest verbunden, die Stege (1,2 mm = 3 Düsenbreiten) werden als eigene Bahnen gedruckt.



oberste Schicht (z = 1 mm) aus dem G-Code, 5 Schichten, geschätzte Druckzeit 53m 3s, Filament 2058.04 mm

7 Wie eine Glyphe entsteht

Beispiel B: Stamm `vline(l, 0, cap)`; oberer und unterer Bauch als Pfad vom Stamm (`bowl`: Querstrich, Viertelbögen, Querstrich zurück); drei Stege direkt rechts neben dem Stamm (Querstriche oben, Mitte, unten). `region2::stroke`

macht aus den Pfaden Bänder, Gehrungsstücke und Enden, `region2::combine` vereinigt sie und zieht die Stege ab. Ergebnis: Stamm und „3“-förmiger Bauch als zwei geschlossene Konturen aus Geraden und Bögen, ohne Loch. Die Lagen hängen von der Strichstärke ab (Mittellinie = äußere Linie ∓ halber Strich), daher bleiben Versalhöhe und Glyphenbox bei jeder Strichstärke gleich.

```
'B' => {
    g.vline(l, y0, cap);
    bowl(g, l, r - 0.06 * wd, t, m, (t - m) / 2.0);
    bowl(g, l, r, m, b, (m - b) / 2.0);
    for y in [t, m, b] {
        g.cut(l + h + bw / 2.0, y, 90.0);
    }
}
```

8 Messwerte

Kernel-Zeit

Release-Build, `cad eval --json` ohne Daemon, Median aus 3 Läufen; Kernel-Zeit in ms (alle Kernel-Aufrufe). Die Schablonenschrift braucht weniger Kernel-Aufrufe und Zeit als Allerta Stencil und JetBrains Mono: Geraden und Kreise statt B-Splines; ihre eigene Rechenzeit (Regionen, im Cache) steckt in der Auswertung (Sprache 1,6 ms für das Schild).

Modell	cad stencil mono	Allerta Stencil	JetBrains Mono	Schablone / Allerta
„Hello World“, extrudiert	130 (18 Aufrufe)	414 (26 Aufrufe)	417 (20 Aufrufe)	0,31
40 Zeichen, extrudiert	962 (53 Aufrufe)	2.652 (95 Aufrufe)	2.126 (61 Aufrufe)	0,36
Schild: Platte ∪ „ZUTRITT NUR MIT BRILLE“ (size 4,2)	1.791 (33 Aufrufe)	3.419 (45 Aufrufe)	2.238 (34 Aufrufe)	0,52

Browser (Chromium, SMLib als Wasm, Playwright K03): Schild 2.196 ms gegen 3.208 ms mit Allerta Stencil; „Hello World“ 206 gegen 463 ms. Beispiel Schablone nativ: 2.609 ms.

Wasm-Modul

	roh	gzip -9
`cad_wasm_bg.wasm` vorher (Commit ed85270)	4.120.239 B	1.757.290 B
mit Schablonenschrift	4.162.774 B	1.772.667 B
Zuwachs	42.535 B	15.377 B

Gleicher Build (release, wasm-opt -O3) für beide Stände. `region2` war schon im Modul (offset, stroke); hinzu kommen nur die Zeichenfunktionen.

9 OpenSCAD-Kompatibilität

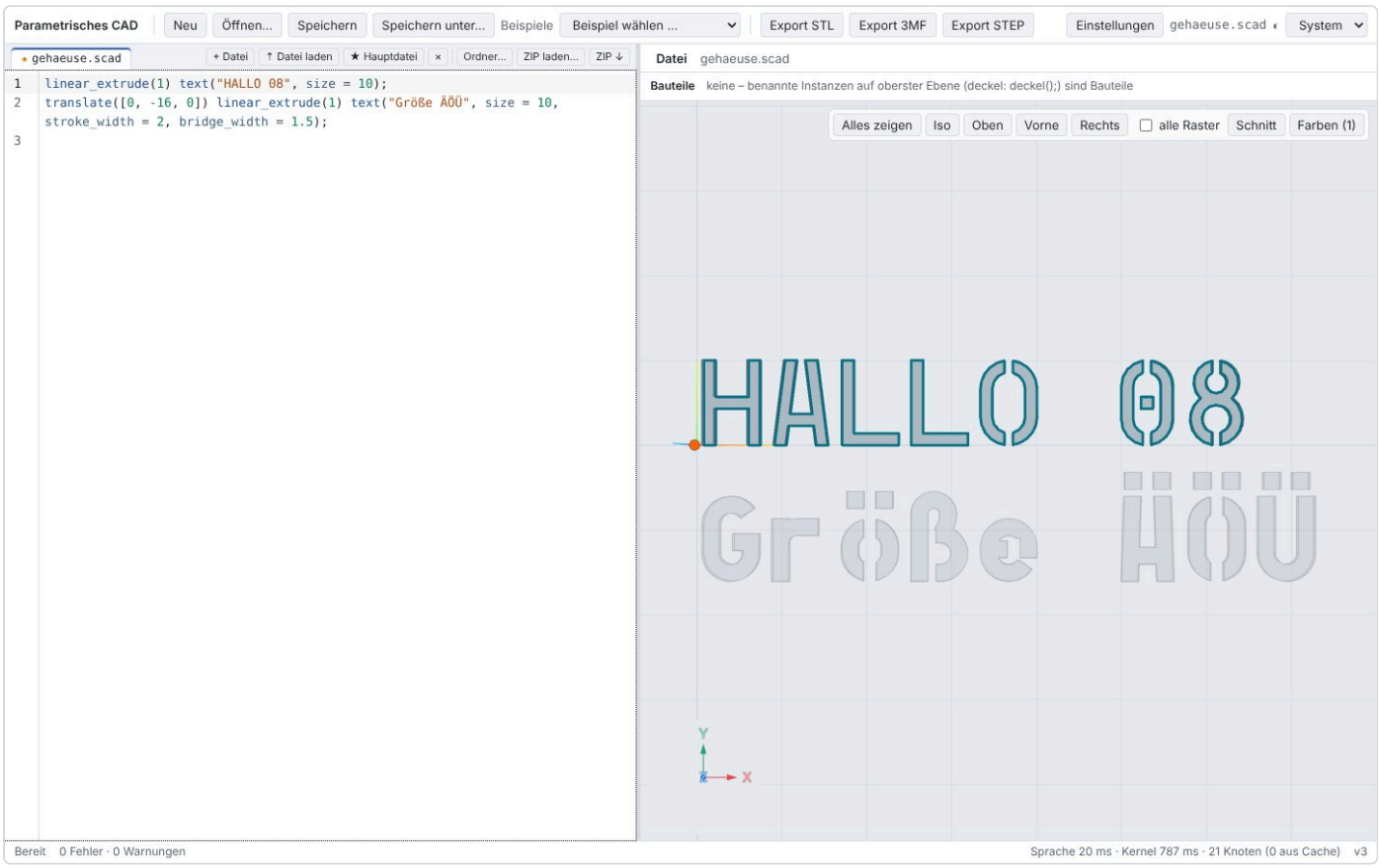
Hängen die Kompatibilitätstests von der Standardschrift ab? Nein: Die 36 Vergleichsmodelle (`examples/openscad-compat`, `tests/openscad-compat/projects`) enthalten kein `text()` (grep findet keine Datei), `compare.py` und die Stufe-1-Tests (`crates/compat-tests`) sind damit unberührt; der Textvergleich mit OpenSCAD aus Bericht 16 (`hull_and_area_like_openscad`, `report16-openscad.py`) nennt jede Schrift ausdrücklich. Lauf `compare.py --eval`: 36/36 files match. Bewusste Abweichung: `text("…")` ohne `font` ergibt bei uns die Schablonenschrift, in OpenSCAD dessen fontconfig-Standard – für gleiche Maße eine Schrift angeben (Plan 17.11, Skill).

10 UI

Eine Schriftauswahl gibt es in der UI nicht (Schriften stehen im Code; eigene per „⇡ Datei laden“), daher dort keine Änderung. Neue Szenarien in `web/e2e/schablone.spec.ts`:

K01 Standardschrift im Browser

Die Wasm-Engine nutzt dieselbe Schrift wie die CLI: Versalhöhe = size, Vorschub wie JetBrains Mono, Umlaute ohne Rückfall.



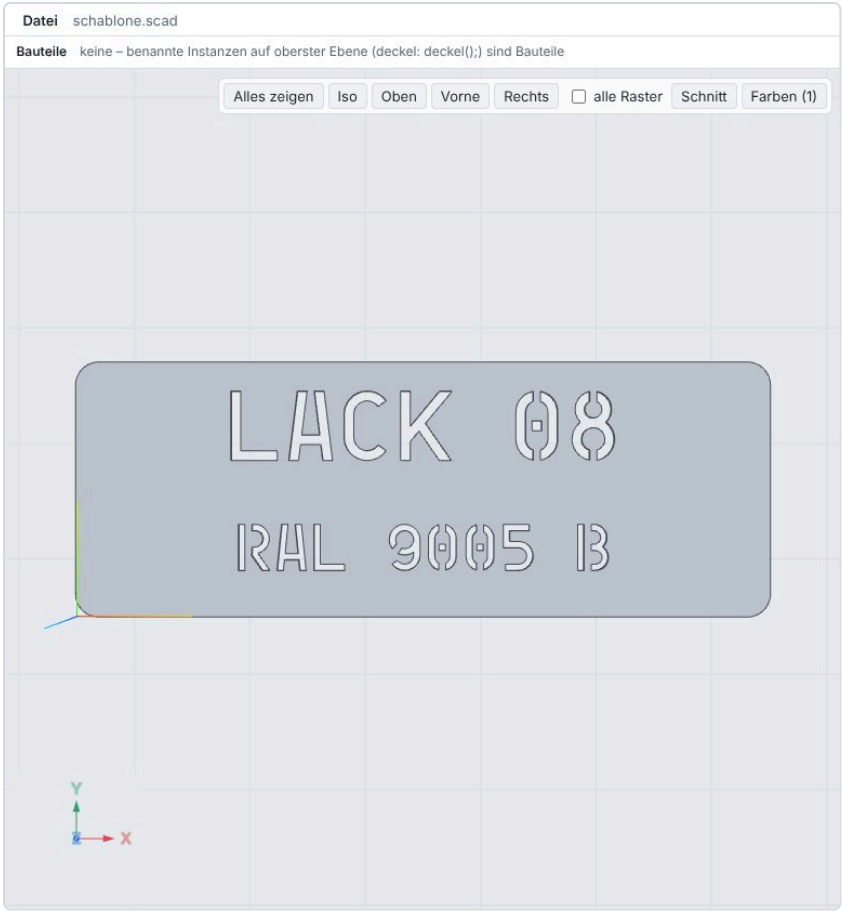
text() ohne font: cad stencil mono, zweite Zeile mit stroke_width = 2, bridge_width = 1.5

```
linear_extrude(1) text("HALLO 08", size = 10);
translate([0, -16, 0]) linear_extrude(1) text("Größe ÄÖÜ", size = 10, stroke_width = 2, bridge_width = 1.5);
```

- keine Meldungen
- Versalhöhe 10 mm, Höhe 1 mm
- Breite 7–9 Zellen
- Grenzen und fremde Schrift: Warnungen

K02 Beispiel Schablone

Platte minus Text: die Innenteile von A, 0, 8, B, 9 hängen über die Stege an der Platte.



examples/spec/schablone.scad im Browser (Draufsicht)

- keine Fehler/Warnungen
- Hülle 120 × 44 × 1 mm

C16 achsgebundenes Ziehen (Bericht 4) – Ursache und Korrektur

C16 lief seit Bericht 15 zeitabhängig in den Test-Timeout, zuletzt auch einzeln. Ursache: Nach `clickLine(2, 3)` baut der Viewer die Griffe erst im nächsten Animationsframe neu; bis dahin liefert `__cad.handles()` die Griffe der vorigen Cursorzeile. Dort steht `cube([60, 30, 1])`, dessen Vektor ebenfalls ein Punktgriff ist – `waitHandles("… kind === 'point'")` war sofort erfüllt, der Test nahm den ersten Griff (die Plattenecke bei x = 1118 px statt des translate-Punkts bei 770 px), das Ziehen begann nicht (`drag() = null`), die Maus drehte die Kamera, und das Warten auf „Achse x“ lief bis zum Timeout. Die Achswahl selbst war in allen Läufen richtig. Korrektur im Test: `pointHandleOn(page, zeile)` wartet auf den Punktgriff **der Zeile** und nimmt genau diesen (ebenso C20, dieselbe Lücke). Kein Test übersprungen.

```
vorher (pnpm exec playwright test e2e/ui-c.spec.ts -g C16, mehrere Läufe mit --repeat-each): 10 von 16
bestanden, 6 × Test-Timeout (waitForFunction drag()?.axis === 0)
  fehlgeschlagener Lauf: Griff bei Wartebeginn {"line":1,"value":[60,30,1],"screen":{"x":1117.7,"y":527.9}}
(cube der Zeile 1), drag() = null
nachher (pointHandleOn): C16 16 von 16, C20 8 von 8 bestanden (je ca. 25 s bzw. 10 s)
```

11 Testläufe

Lauf	Ergebnis
`cargo test --workspace --no-fail-fast`	529 bestanden, 0 fehlgeschlagen
`cargo clippy --workspace --all-targets -- -D warnings` · `cargo fmt --check`	sauber · sauber
`pnpm typecheck` · `pnpm test` (Vitest)	sauber · 221 bestanden
`pnpm e2e` (Playwright, Chromium)	106 bestanden, 0 fehlgeschlagen

`tests/openscad-compat/compare.py --eval`	36/36 files match

Neue bzw. geänderte Rust-Tests (13):

`every_glyph_profile_has_no_holes` **ok** · `extruded_text_volume` **ok** · `sign_text_kernel_time_like_allerta` **ok** · `text_stencil_parameters` **ok** · `text_profiles_and_fonts` **ok** · `tests::font_names` **ok** · `tests::stencil_areas` **ok** · `tests::required_chars_in_every_font` **ok** · `tests::stencil_layout_features` **ok** · `tests::stencil_metrics_and_monospaced_advance` **ok** · `tests::stencil_without_bridges_has_counters` **ok** · `tests::stencil_glyphs_have_no_holes` **ok** · `spec_models_stage1` **ok**

cad-font: Abdeckung (`required_chars_in_every_font` mit beiden Stilen), keine Löcher/Splitter/Überlappungen für 8 Parameterkombinationen, geschlossene Innenräume ohne Stege, Metrik (Versalhöhe = size, Vorschub = JetBrains Mono, jede Glyphe in ihrer Zelle), Flächen (H exakt, o gegen Formel, Alphabet gegen JetBrains Mono, Strich/Steg/Bold/size), Ausrichtung, Richtungen, Rückfall, Schriftnamen. cad-eval: Standardschrift, Parameter, Warnungen. cad-engine: jede Glyphe als Profil ohne Loch (Regular, Bold), Volumen = Fläche × Höhe, Kernel-Zeit des Schilds gegen Allerta Stencil. compat-tests: Beispiel `schablone.scad`.

12 Grenzen

- Gestaltung bewusst einfach: eine Strichstärke je Glyphe (nur @ mit 0,72-fachem Strich, ² ³ mit 0,75-fachem), keine optischen Korrekturen, keine Kerning-Paare (monospaced). W, w, m, @, * werden bei Bold sehr dicht.
- Grenzen der Parameter aus dem Test (Strich ≤ 0,2·size, Steg 0 oder 0,03…0,2·size); dazwischen geprüft an 8 Kombinationen, nicht lückenlos. Ein Steg ist an Bögen radial gerade geschnitten, seine Breite gilt auf der Mittellinie.
- Kombinierende Zeichen (z. B. „a“ + U+0308) setzt die Schrift nicht zusammen: das Zeichen kommt aus JetBrains Mono. Vorkomponierte Umlaute sind enthalten.
- Zeichen außerhalb des Umfangs (Kyrillisch, Griechisch, Pfeile …) kommen aus JetBrains Mono – ohne Stege, können also Löcher haben.
- Glyphen verschiedener Zeichen berühren sich nur bei `spacing` < 1 oder den durchgehenden Strichen –, _ (über die ganze Zelle wie in Monospace-Schriften); dann können zwischen Glyphen Innenräume entstehen.
- Modelle ohne `font` sehen in OpenSCAD anders aus (dort fontconfig-Standard).